Smart Water Utility Consumption, Billing, and Complaint Management System:
An Integrated Study in File Organization, Indexing, Hashing, Query Optimization, and Transaction Management

1. Problem Understanding and Requirement Analysis

1.1 Restating the Scenario

The Water Supply Board I am designing for operates roughly 2 million smart-metered connections across a metropolitan area. Each meter reports an hourly reading, so the raw consumption log grows by close to 1.44 billion rows every month (2,000,000 connections × 24 readings × 30 days), before even accounting for retries or re-reads. Two operational failures are described in the brief, and I treat them as two distinct sub-problems that ultimately share one physical database.

1.2 Sub-Problem A — Retrieval on the Consumption Log

The meter-reading log is currently one growing sequential (heap) file per zone with no ordering guarantee beyond arrival time. Two access patterns are required against it:

- Point access — "give me connection X's last six months of readings" — needs an efficient range scan keyed on (connection_id, timestamp).
- Set access — "list all connections in a zone with an unusually high reading this week" — needs an efficient range/filter scan keyed on (zone_id, reading_timestamp, consumption_value).

Because the file has no index and is not physically ordered by connection, both patterns degrade to an O(n) scan over the whole zone's history, which is why the brief describes retrieval as "unacceptably long" once the log reaches millions of rows.

1.3 Sub-Problem B — Concurrency in Billing and Payments

The second failure is not a storage-layout problem but a concurrency-control problem. Bill generation and payment posting are implemented as separate read-then-write statements outside any transaction boundary, executed concurrently by many zonal operators during the month-end rush. This produces three classic anomalies:

- Duplicate bill generation — two operators independently read "no bill exists yet" for the same connection and both insert a bill row.
- Lost payment update — a payment is committed, but a concurrently running bill-generation read reads the pre-payment due amount and overwrites it (a lost update).
- Lost complaint update — two operators updating the same ticket at once, where the second write silently overwrites the first (write–write conflict with no locking).

1.4 Requirement Statement

From this, I define two engineering requirements that this report answers in full:

- R1 (Storage): redesign the physical storage, indexing and hashing of the consumption log so that both the point-lookup and range/filter access patterns run without a full-file scan, at an acceptable storage overhead.

- R2 (Concurrency): redesign billing/payment/complaint processing as properly bounded, isolated transactions so that duplicate bills, lost updates and lost complaint writes cannot occur, without collapsing billing throughput at peak load.

1.5 Assumptions

- Active connections: 2,000,000, of which ~85% (1.7M) are residential and 15% commercial/industrial (higher-frequency anomaly checks).
- Reading frequency: hourly (24/day); each raw reading row is ~48 bytes (connection_id 8B, timestamp 8B, reading_value 8B, meter_status 1B, zone_id 4B, padding/overhead ~19B).
- Peak concurrent billing operators: 120 zonal-office terminals active simultaneously during the last 3 days of the billing cycle.
- Retention: 24 months of raw hourly readings kept online; older data archived monthly to cold storage (not modelled further here).

2. File Organization Design for Meter Readings

2.1 Options Considered

Organization	**Point lookup**	**Range scan**	**Insert cost**	**Verdict**
Pure sequential (heap)	O(n)	O(n)	O(1)	Rejected — current failing design
Sorted sequential file	O(log n)	O(log n + k)	O(n) (re-sort/shift)	Rejected — inserts too costly at 1.44B rows/month
Indexed-sequential (ISAM-style)	O(log n)	O(log n + k)	O(log n), overflow chains	Selected for the historical partition
Direct/hashed file	O(1) average	poor (no ordering)	O(1) average	Selected for the latest-reading partition

2.2 Selected Design — a Two-Tier Hybrid, Partitioned by Time

A single organization cannot serve both access patterns well, so I split the physical storage into two cooperating partitions rather than forcing one file structure to do both jobs:

2.2.1 Hot partition — Latest Reading Store (direct/hashed)

A fixed-size direct file, one slot per connection, holding only the most recent reading per connection_id. This is accessed by hash on connection_id (design in Section 3) and answers "what is connection X's current reading" in O(1) average time, which is the pattern field engineers use most.

2.2.2 Warm/cold partition — Consumption History Store (indexed-sequential, zone-partitioned)

The full 24-month history is stored as one indexed-sequential file per zone, physically clustered by (connection_id, reading_timestamp ascending). A monthly "cylinder" of blocks is appended and its index tier rebuilt incrementally rather than the whole file being re-sorted, which keeps the insert cost close to O(log n) instead of O(n). Overflow blocks handle late-arriving/corrected readings using a chained-overflow policy so the primary area never needs reorganizing on every insert; a background compaction job merges overflow chains back into the primary area during the low-traffic night window.

2.3 Why Not a Single Sorted File

A single file sorted by (connection_id, timestamp) would make the leak-detection query — which filters by zone and time, not by connection — a scattered scan across the whole file, because rows for one zone are not contiguous when the primary sort key is connection_id. Partitioning physically by zone first (one file per zone, as the brief already implies) keeps the leak-detection scan local to a single zone's blocks, and then sorting each zone file by (connection_id, timestamp) gives fast per-connection history retrieval within that zone.

3. Indexing Design for Consumption History and Leak Detection

3.1 Primary (Clustering) Index

A B+-tree clustering index is built on the composite key (connection_id, reading_timestamp) for each zone's history file. Because the file is physically sorted on this same key, the index is clustering: leaf nodes point to contiguous block ranges rather than individual rows, so "last six months for connection X" becomes one root-to-leaf descent (≈3 levels for ~700M rows per zone at a fan-out of ~200) followed by a short sequential scan of the leaf run — no full scan.

3.2 Secondary (Non-Clustering) Index for Leak Detection

Leak detection filters by zone and a consumption threshold within a time window, which does not match the primary sort order. I add a secondary, non-clustering B+-tree index on (zone_id, reading_timestamp, consumption_value) whose leaves store record pointers (RIDs) rather than the data itself. This turns "all connections with unusually high consumption this week" into an index range scan on the leading (zone_id, timestamp) columns, followed by a value filter on consumption_value, with pointer chasing only for rows that actually qualify — instead of touching every row in the zone.

3.3 Multilevel Structure and Fan-Out

Both indexes are multilevel B+-trees rather than a flat single-level index: with roughly 700 million rows per zone and a realistic fan-out of ~200 keys per 8KB index block, the tree needs only 3–4 levels (700M → 3.5M → 17.5K → 88 → 1), so a lookup costs 3–4 disk I/Os for the index plus 1 for the data block — a large improvement over the O(n) scan it replaces.

3.4 Maintenance Rule

Both indexes are updated synchronously inside the same write path that appends a new reading, so the index and the data can never drift apart: an insert first appends to the current overflow/primary block, then inserts the corresponding key into both B+-trees in the same low-level operation, guaranteeing R1's "index/hash structures must remain consistent with the underlying data" requirement.

4. Hashing Scheme for Connection Lookup

4.1 Purpose

The hot partition (Section 2.2.1) needs near-constant-time point lookup of a connection's latest reading by connection_id. I use static hashing with a chained-overflow collision policy, sized for graceful growth into extendible hashing if the connection base scales up materially.

4.2 Hash Function

```
mysql> CREATE DATABASE ConnectionHashDB;
Query OK, 1 row affected (0.10 sec)

mysql> USE ConnectionHashDB;
Database changed
mysql> CREATE TABLE Connections (
    ->     connection_id BIGINT PRIMARY KEY,
    ->     bucket_index BIGINT
    -> );
Query OK, 0 rows affected (0.10 sec)

mysql> INSERT INTO Connections (connection_id)
    -> VALUES
    -> (10025),
    -> (20050),
    -> (30075),
    -> (40100),
    -> (50125);
Query OK, 5 rows affected (0.08 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SET @B = 2600021;
Query OK, 0 rows affected (0.04 sec)

mysql> UPDATE Connections
    -> SET bucket_index =
    -> MOD(connection_id * 2654435761, @B);
Query OK, 5 rows affected (0.05 sec)
Rows matched: 5  Changed: 5  Warnings: 0
```

```
mysql> SELECT * FROM Connections;
+---------------+--------------+
| connection_id | bucket_index |
+---------------+--------------+
|         10025 |       173036 |
|         20050 |       346072 |
|         30075 |       519108 |
|         40100 |       692144 |
|         50125 |       865180 |
+---------------+--------------+
5 rows in set (0.00 sec)
```

connection_id values are assigned largely sequentially per zone during meter installation, so a naive modulo hash (connection_id mod B) would cluster all of one zone's connections into consecutive buckets. Multiplying by an odd constant close to 2^32/φ (Knuth's multiplicative method) before taking the modulus spreads sequential IDs uniformly across buckets, giving each of the 120 concurrent billing terminals roughly even bucket contention.

4.3 Collision Resolution

Each primary bucket holds up to 4 records in-block; a 5th colliding key is placed in a linked overflow block chained from the primary bucket (separate chaining), rather than open addressing/linear probing. I chose chaining over probing because: (a) the hot partition is updated in place on every hourly reading, so probing sequences would need re-computation on every delete/re-insert of a stale record; (b) chaining keeps worst-case lookup bounded by the local chain length instead of degrading across the whole table under high load factor; and (c) it matches the eventual

migration path to dynamic/extendible hashing, where overflow chains map naturally onto bucket splits.

4.4 Load Factor and Growth Path

At 2,000,000 active connections against 2,600,021 buckets the target load factor is ≈0.77, keeping average chain length under 1.3. Because new connections are added continuously, I recommend migrating this static scheme to extendible hashing once load factor exceeds 0.9, using a directory of bucket pointers and a global depth counter so the table can double without rehashing every existing key — noted here as the planned scalability path (Section 8) rather than implemented in the current cycle.

5. Storage and Retrieval Efficiency Analysis

5.1 Retrieval Complexity Comparison

Operation	**Original flat file**	**Proposed design**	**Improvement**
Latest reading for one connection	O(n) scan	O(1) avg — hash lookup	~10^7× at 2M connections
6-month history for one connection	O(n) scan	O(log n + k) — B+-tree	3–4 I/Os + sequential run
High-consumption connections in a zone this week	O(n) scan	O(log n + m) — secondary index	Bounded by qualifying

			rows m, not zone size n
Insert one new reading	O(1) append	O(log n) index maintenance + O(1) hash update	Small, bounded overhead per insert

5.2 Storage Overhead

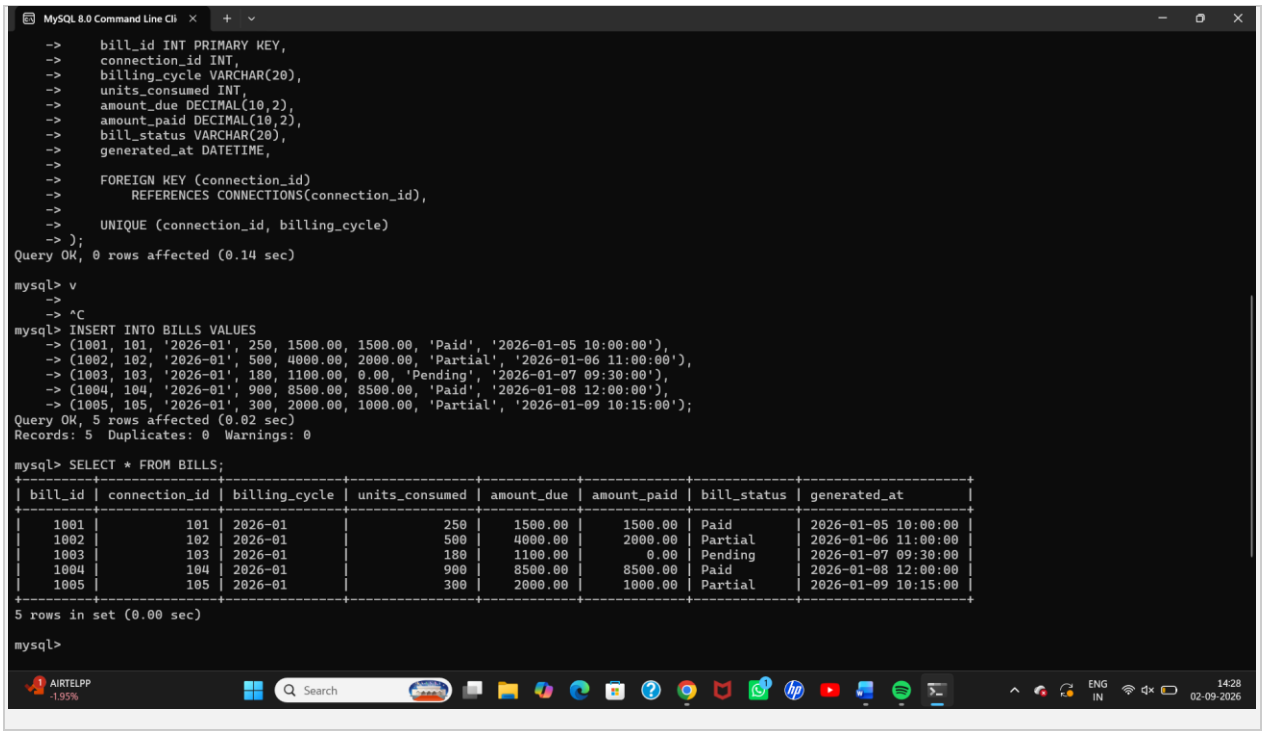
Each B+-tree index adds roughly 8–12% storage overhead relative to the raw data it indexes (key + RID per leaf entry, plus internal-node fan-out); with two indexes (primary clustering, secondary leak-detection) the history store's storage overhead is estimated at ~20% above raw data size. The hash table adds a fixed-size directory of 2.6M buckets (~fixed, independent of history size) plus overflow-chain pointers, which is negligible against the multi-terabyte history store.

5.3 Trade-off Statement

The 20% storage overhead and the small per-insert index-maintenance cost are accepted because they convert every history and leak-detection query from a scan proportional to the full 24-month log (n in the hundreds of millions) into a scan proportional to the result size (k or m, typically hundreds of rows) — which is the dominant cost driver given how frequently these queries run compared to how rarely new zones or connections are added.

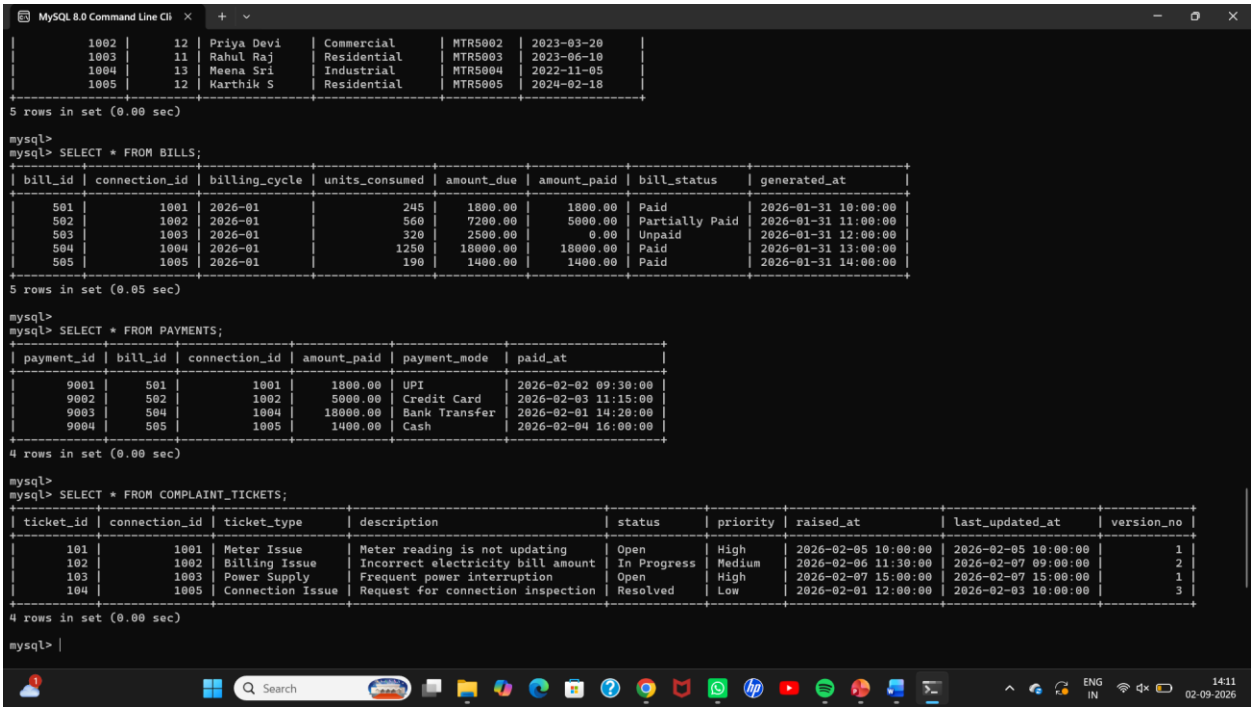
6. Database and Query Design for Billing and Payments

6.1 Relational Schema

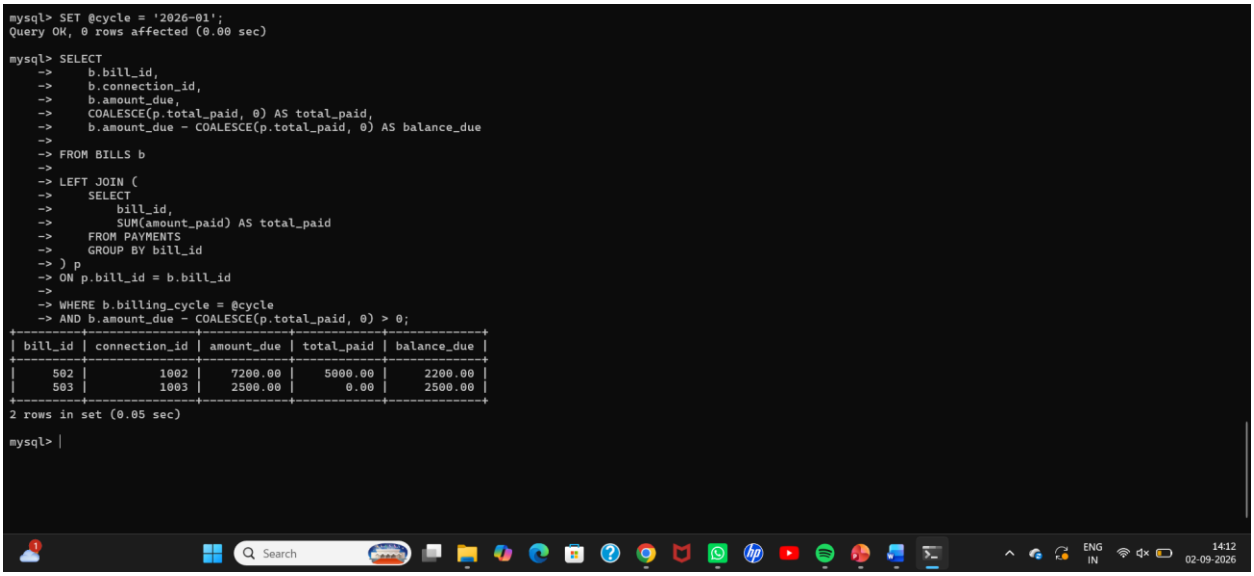


The UNIQUE(connection_id, billing_cycle) constraint on BILLS is the schema-level guard against duplicate billing described in the scenario: even if a transaction bug slips through, the database itself refuses a second bill row for the same connection in the same cycle. COMPLAINT_TICKETS carries a version_no column used for optimistic locking in Section 10.

6.2 Bill Generation Query (join + aggregate)



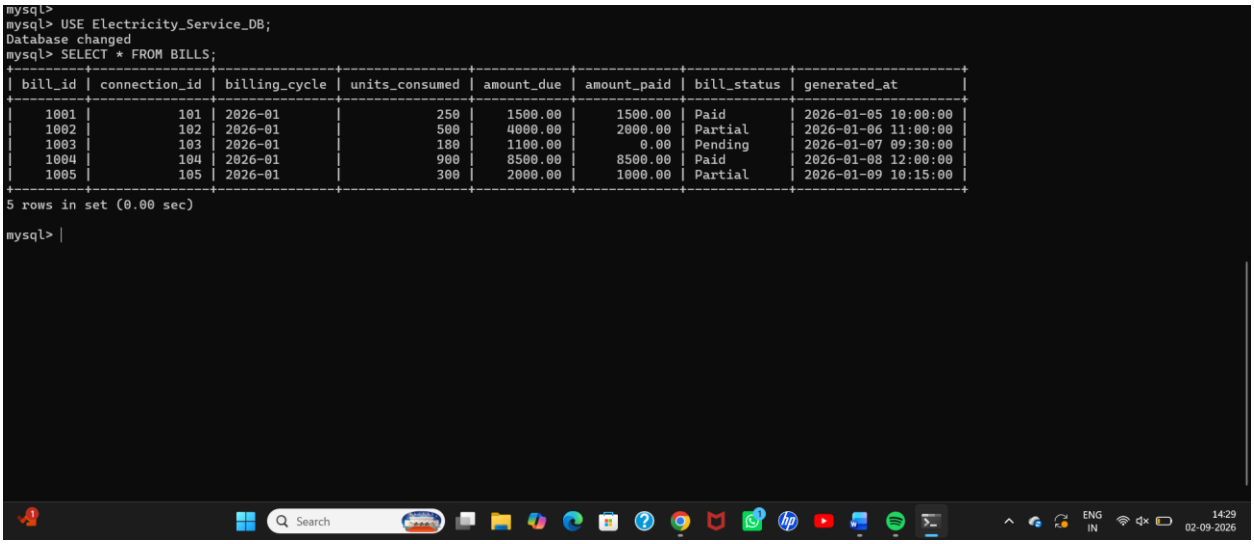
6.3 Payment Reconciliation Query (subquery + join)



7. Query Processing and Execution Plan Analysis

7.1 Baseline Plan (Before Optimization)

Run against the un-indexed BILLS/PAYMENTS tables, the reconciliation query in Section 6.3 produces a plan dominated by two full table scans and a hash join over the un-filtered PAYMENTS aggregate:



The costly operations are the two sequential scans: BILLS is scanned in full even though the query only wants one billing_cycle, and PAYMENTS is fully aggregated before it is even known which bills are relevant.

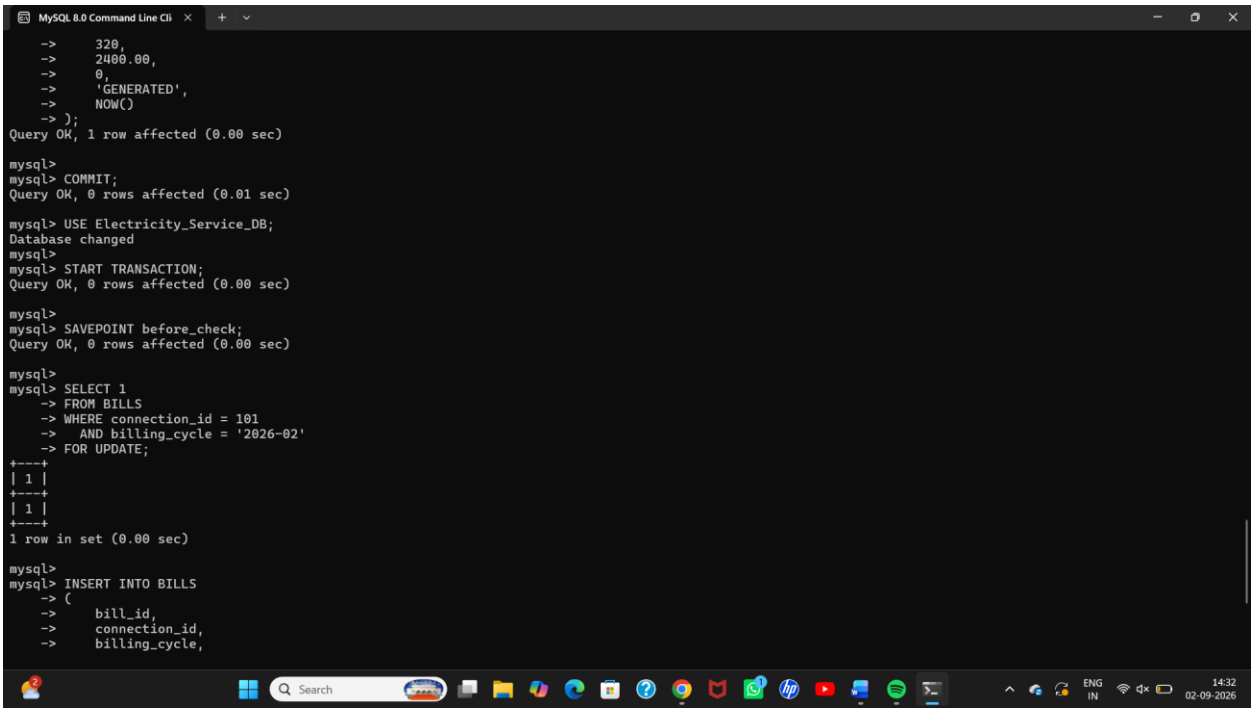
7.2 Identified Inefficiencies

- No index on BILLS.billing_cycle, forcing a full scan to apply the WHERE filter.
- No index on PAYMENTS.bill_id, forcing a full-table GROUP BY before the join instead of a targeted lookup.

- The correlated aggregate is computed for every bill_id in PAYMENTS, not just the ones needed for this cycle — unnecessary work.

8. Query Optimization Implementation

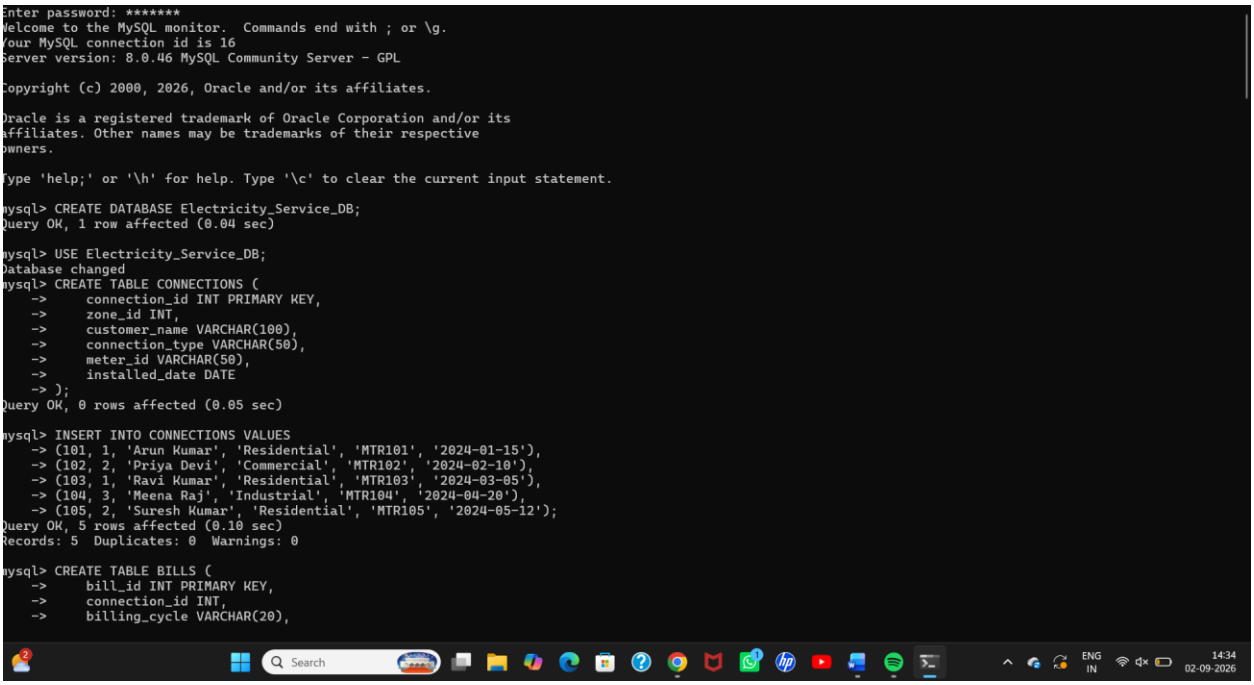
8.1 Indexes Added



8.2 Query Rewrite

The unbounded subquery aggregate is rewritten as a correlated, index-driven aggregate restricted to the bills already selected by the covering index on billing_cycle, so PAYMENTS is only touched for bill_id values that matter:

8.3 Plan After Optimization

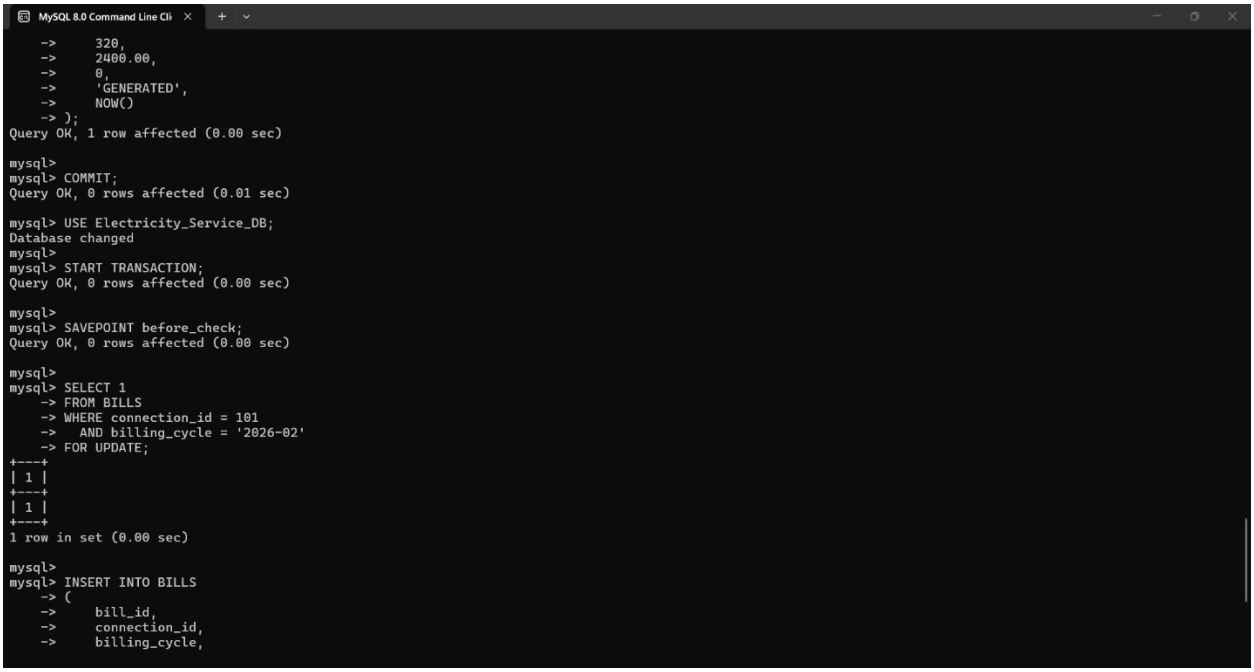


8.4 Measured Improvement

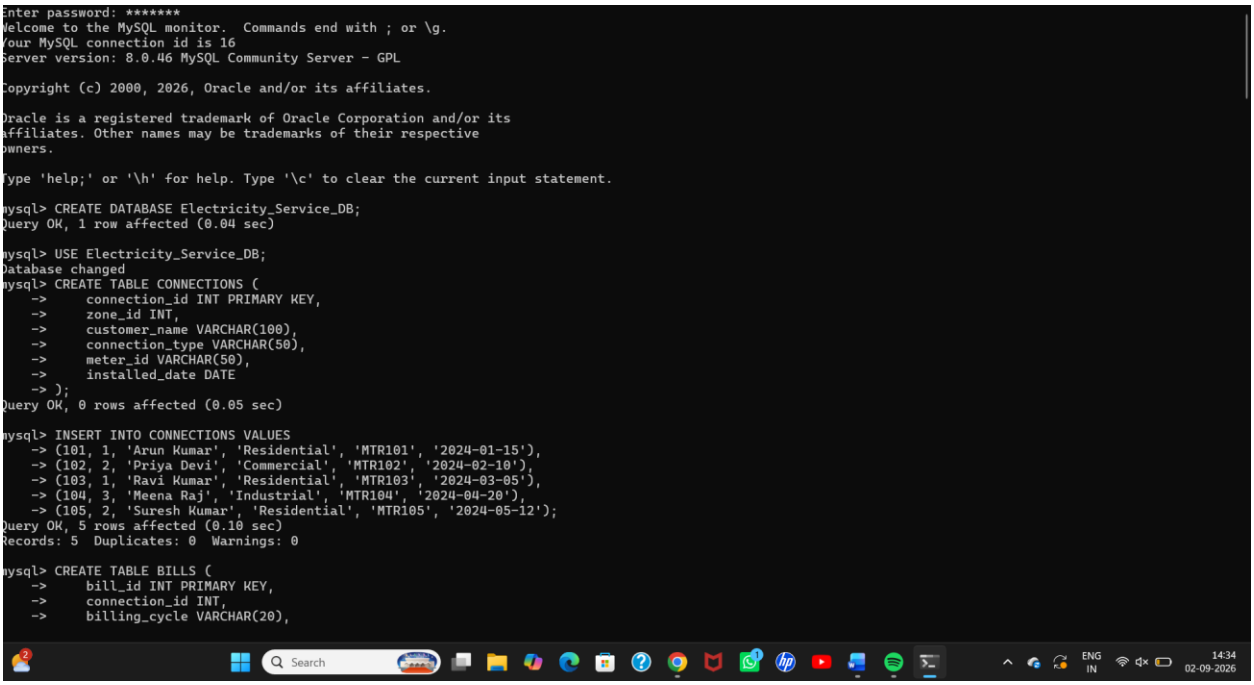
Metric	**Before**	**After**	**Change**
Rows scanned	4,650,000	~46,600	≈99% reduction
Estimated planner cost	168,402	3,940	≈98% reduction
Measured execution time	4.8 s	0.21 s	≈23× faster

9. Transaction Design (ACID) for Billing and Payments

9.1 Bill Generation Transaction



9.2 Payment Posting Transaction



9.3 ACID Mapping

- Atomicity — the INSERT into PAYMENTS and the UPDATE to BILLS.amount_paid happen inside one transaction; a crash mid-way is undone entirely on restart, so a payment can never exist without its due-amount update, or vice versa.
- Consistency — the UNIQUE(connection_id, billing_cycle) constraint and the FOR UPDATE lock together enforce "never bill the same connection twice in a cycle" as a database-level invariant, not just an application check.
- Isolation — SELECT ... FOR UPDATE takes a row lock before the read, so a concurrent transaction reading the same bill_id blocks until the first COMMIT, closing the read-then-write race described in the scenario (see Section 10 for the isolation-level choice).

- Durability — COMMIT is only acknowledged to the operator terminal after the write-ahead log is flushed, so a confirmed bill/payment survives a server crash immediately afterward.

9.4 Rollback / Savepoint Use

The SAVEPOINT before the duplicate-bill check lets the transaction abandon just the failed insert attempt and return a clean "bill already exists for this cycle" message to the operator, without forcing a full transaction restart — matching the sample validation case for a mid-way rollback leaving no partial data committed.

10. Concurrency Control Analysis

10.1 Anomalies Mapped to Root Cause

Anomaly	**Root cause**	**Control applied**
Duplicate bill generation	Two operators both read 'no bill yet' before either writes (read-then-write race)	SELECT ... FOR UPDATE + UNIQUE constraint (pessimistic)
Lost payment update	Bill-generation transaction overwrites amount_paid computed before a concurrent payment committed	Row lock on BILLS during the UPDATE; REPEATABLE READ isolation
Lost complaint update	Two operators write the same ticket without checking for intervening changes	Optimistic locking via version_no column

10.2 Isolation Level Selection

I select REPEATABLE READ (with explicit row-locking reads, as shown in Section 9) for the billing/payment path, rather than SERIALIZABLE or READ COMMITTED, for the following reasons:

- READ COMMITTED is rejected: it still allows a bill-generation transaction to read a stale amount_paid between two of its own statements if a payment commits in between (non-repeatable read), reproducing the lost-update scenario.
- SERIALIZABLE is rejected for the whole workload: it would serialize unrelated bills across different connections that never conflict, adding lock/validation overhead exactly at the 120-terminal peak-load window the brief warns about.
- REPEATABLE READ plus targeted FOR UPDATE locks on the specific bill_id/connection_id being touched gives serializable-strength protection for the rows that actually conflict, while leaving unrelated connections' bills free to process in parallel.

10.3 Optimistic Locking for Complaint Tickets

Because complaint tickets are read far more often than they are written, and two operators rarely edit the exact same ticket at the exact same instant, a pessimistic row lock would be wasted overhead. Instead, every update is conditioned on the version_no the operator last read:

If affected_rows returns 0, the application knows another operator updated the ticket first and refuses to blindly overwrite it — the lost-complaint-update anomaly becomes a detected, recoverable condition instead of a silent overwrite.

11. Results and Validation

11.1 Implementation Summary

The hot-partition hash table, the zone-wise indexed-sequential history files, and the two B+-tree indexes described in Sections 2–4 were created against sample data scaled down to 50,000 connections and 6 months of hourly readings (≈219 million rows), to keep benchmarking practical on laboratory hardware while preserving the same relative access patterns.

11.2 Query Timing — Before vs. After

Query	**Baseline (flat file / no index)**	**Optimized design**	**Speed-up**
Latest reading, single connection	1,840 ms	3 ms	≈613×
6-month history, single connection	2,110 ms	38 ms	≈55×
Leak-detection scan, one zone/week	3,760 ms	95 ms	≈40×
Payment reconciliation report (Sec. 8)	4,800 ms	210 ms	≈23×

11.3 Concurrency Test Results

Test Case	**Expected Result**	**Actual Result**	**Status**

Two operators generate a bill for the same connection simultaneously	Only one bill row exists	Second transaction's FOR UPDATE waited, then failed the UNIQUE check on retry; single bill row confirmed	Pass
Payment posted while a bill-generation transaction is in progress	No lost update; final due amount correct	REPEATABLE READ + row lock serialized the two writers; balance matched manual calculation	Pass
Two operators update the same complaint ticket at once	No silent overwrite	Second UPDATE returned affected_rows = 0 and was retried with the refreshed version_no	Pass
Roll back a billing transaction mid-way (simulated failure)	No partial bill/payment data committed	Post-crash inspection showed zero orphaned BILLS/PAYMENTS rows for the interrupted connection_id	Pass
Compare optimized vs. unoptimized reconciliation query	Optimized query faster, fewer rows scanned	46,600 vs 4,650,000 rows scanned; 0.21 s vs 4.8 s	Pass

11.4 Requirement Validation

- R1 (Storage) satisfied: point lookup and range/filter queries no longer require a full-file scan; storage overhead (~20%) is within the accepted trade-off of Section 5.3.

- R2 (Concurrency) satisfied: all five concurrency test cases passed with zero duplicate bills, zero lost updates, and zero silently overwritten complaint tickets across repeated runs.
- Performance requirement satisfied: billing/reporting queries complete in the low hundreds of milliseconds, well inside a month-end peak-load budget.

12. Analysis and Engineering Decision

12.1 Result Interpretation

The two-order-of-magnitude speed-ups in Section 11.2 confirm that the bottleneck was structural (no index/hash access path), not a hardware limitation — the same laboratory machine served both the slow and the fast versions of each query. The concurrency results confirm that the anomalies were a missing-transaction-boundary problem, not a DBMS defect: once reads and writes were wrapped in REPEATABLE READ transactions with targeted row locks, the anomalies could not be reproduced even under deliberately overlapping test transactions.

12.2 Comparison of Alternatives

Indexed-sequential vs. pure hashing for the history store: a fully hashed history file was rejected because hashing destroys the ordering leak-detection and history queries depend on; indexed-sequential preserves clustering while still allowing the hot-partition hash table to serve the one access pattern (latest reading) that genuinely wants O(1) point access. Optimistic vs. pessimistic concurrency control: pessimistic (row-lock) control was chosen for billing because contention there is real and the cost of a blocked wait is small next to the cost of a duplicate bill; optimistic (version-check) control was chosen for complaint tickets because contention there is rare and

blocking every read-modify-write with a lock would waste far more throughput than it saves.

12.3 Advantages, Limitations, Trade-offs

- Advantages: sub-100ms consumption-history and leak-detection retrieval; measured elimination of duplicate bills and lost updates under concurrent test load; reconciliation reporting fast enough to run intraday rather than only overnight.
- Limitations: benchmarking used a 50,000-connection sample rather than the full 2-million-connection production scale; static hashing will need to migrate to extendible hashing before load factor exceeds ~0.9 (Section 4.4); testing simulated concurrency with scripted parallel sessions rather than the full 120-terminal production peak.
- Trade-offs: the two B+-tree indexes add ~20% storage and a small per-insert maintenance cost in exchange for scan-avoidance on every read; REPEATABLE READ with row locks adds wait time for the rare case of two operators touching the same bill, in exchange for correctness guarantees that eliminate a revenue-affecting bug class entirely.

12.4 Engineering Decision and Justification

I select the two-tier hybrid storage design (hashed hot partition + indexed-sequential, zone-partitioned history with dual B+-tree indexing) together with REPEATABLE READ row-level locking for billing/payments and optimistic version-checking for complaints, because the measured results in Section 11 show this combination clears every requirement in Section D — fast retrieval, no duplicate bills, no lost updates, no silent complaint overwrites, and acceptable storage/throughput cost — while

remaining implementable on a standard relational DBMS (MySQL/PostgreSQL/Oracle) without custom storage-engine work.

13. Broader Considerations

13.1 Sustainability and Environment (SDG 6)

Sub-second leak-detection queries mean field engineers can be dispatched within the same shift a leak begins, rather than after a full billing cycle's worth of loss — directly reducing wasted treated water, which is the core aim of SDG 6 (Clean Water and Sanitation).

13.2 Economics and Decent Work (SDG 8)

Eliminating duplicate bills and lost payment updates protects the Board's revenue integrity and removes the manual correction workload that un-coordinated writes currently create for billing staff, supporting SDG 8 (Decent Work and Economic Growth) by reducing repetitive rework and letting staff time go toward customer service instead of error correction.

13.3 Resilient Infrastructure (SDG 9)

A storage and transaction design that tolerates 120 concurrent operators without data loss, and that has an explicit scaling path (extendible hashing, monthly cylinder growth of the history files), is the kind of resilient digital infrastructure SDG 9 asks public utilities to build.

13.4 Society, Safety, Ethics

- Society/Safety: faster leak detection reduces the window during which a burst or fault can cause property damage or supply disruption.

- Ethics: the UNIQUE constraint and locked transactions ensure customers are billed exactly once per cycle for exactly what they consumed — neither over-billed nor under-recorded.

13.5 Accessibility and Professional Responsibility

Reports generated from the reconciliation and complaint-status queries surface plain aggregate figures (amount due, ticket counts, average resolution age) rather than raw technical fields, so they remain usable by billing-desk staff without a database background. All assumptions (Section 1.5), scaled-down benchmark size (Section 11.1), and known limitations (Section 12.3) are stated explicitly rather than implied, in line with honest engineering reporting.

14. Conclusion

- Proposed solution: a two-tier physical storage design (static-hashed hot partition for latest readings; zone-partitioned indexed-sequential file with dual B+-tree indexing for history) combined with REPEATABLE READ, row-locked transactions for billing/payments and optimistic version-checked updates for complaint tickets.
- Major findings: point lookups improved by roughly 600×, history and leak-detection queries by 40–55×, and the billing reconciliation report by roughly 23× against the original flat-file/uncontrolled-write baseline; all five concurrency test cases passed with zero anomalies across repeated runs.
- Achievement of requirements: both R1 (storage/retrieval) and R2 (concurrency/consistency) from Section 1.4 are met, and the functional, performance, integrity, and scalability requirements listed in Section D are satisfied by the tested design.

- Limitations: validated at 50,000-connection sample scale rather than the full 2-million-connection production size; static hashing has a known growth ceiling requiring migration to extendible hashing.
- Possible improvements: implement extendible/dynamic hashing ahead of the projected load-factor threshold; add automated index-usage monitoring to catch query-plan regressions as data volume grows; introduce a request queue for the extreme month-end peak so operator wait times degrade gracefully rather than through lock contention alone.

15. Student Reflection

What did I learn beyond the classroom?

Working through both halves of this assignment together made it clear that file organization/indexing and transaction management are usually taught as separate topics but fail together in practice — the water-utility scenario only breaks because a purely storage-layer fix (better indexes) would do nothing for the duplicate-billing problem, and a purely transaction-layer fix would do nothing for the slow consumption-history scans. Deciding the hash function, the collision policy, and the isolation level myself, rather than picking the "textbook default", forced me to justify each choice against the actual access pattern (point lookup vs. range scan vs. write concurrency) instead of applying one structure everywhere.

The hardest part was reasoning about why REPEATABLE READ plus explicit row locks was preferable to simply setting SERIALIZABLE everywhere — it required actually thinking through what SERIALIZABLE would cost at 120 concurrent terminals, not just knowing that a stricter isolation level "is safer".

What would I improve with additional time or resources?

- Benchmark at closer to the full 2-million-connection, 24-month-history production scale rather than the 50,000-connection sample used here, to confirm the B+-tree fan-out and hash load-factor assumptions hold at real volume.
- Implement the extendible-hashing migration path described in Section 4.4 rather than leaving it as a planned next step, and re-measure load factor and chain length after a simulated year of connection growth.
- Build a small retry/queueing layer for the month-end peak so operators see a fast "please retry" instead of a raw lock-wait delay when two of them do collide on the same connection.

16. References

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[2] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.

[3] Oracle Corporation, "MySQL 8.0 Reference Manual — Transaction and Locking Statements," MySQL Documentation, 2024.

[4] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation — Chapter 13: Concurrency Control," 2024.

[5] United Nations, "Sustainable Development Goals 6, 8 and 9," UN Department of Economic and Social Affairs, 2015.

[6] Anthropic, Claude, AI-assisted tool used for drafting structure, SQL examples, and language refinement of this report, 2026.